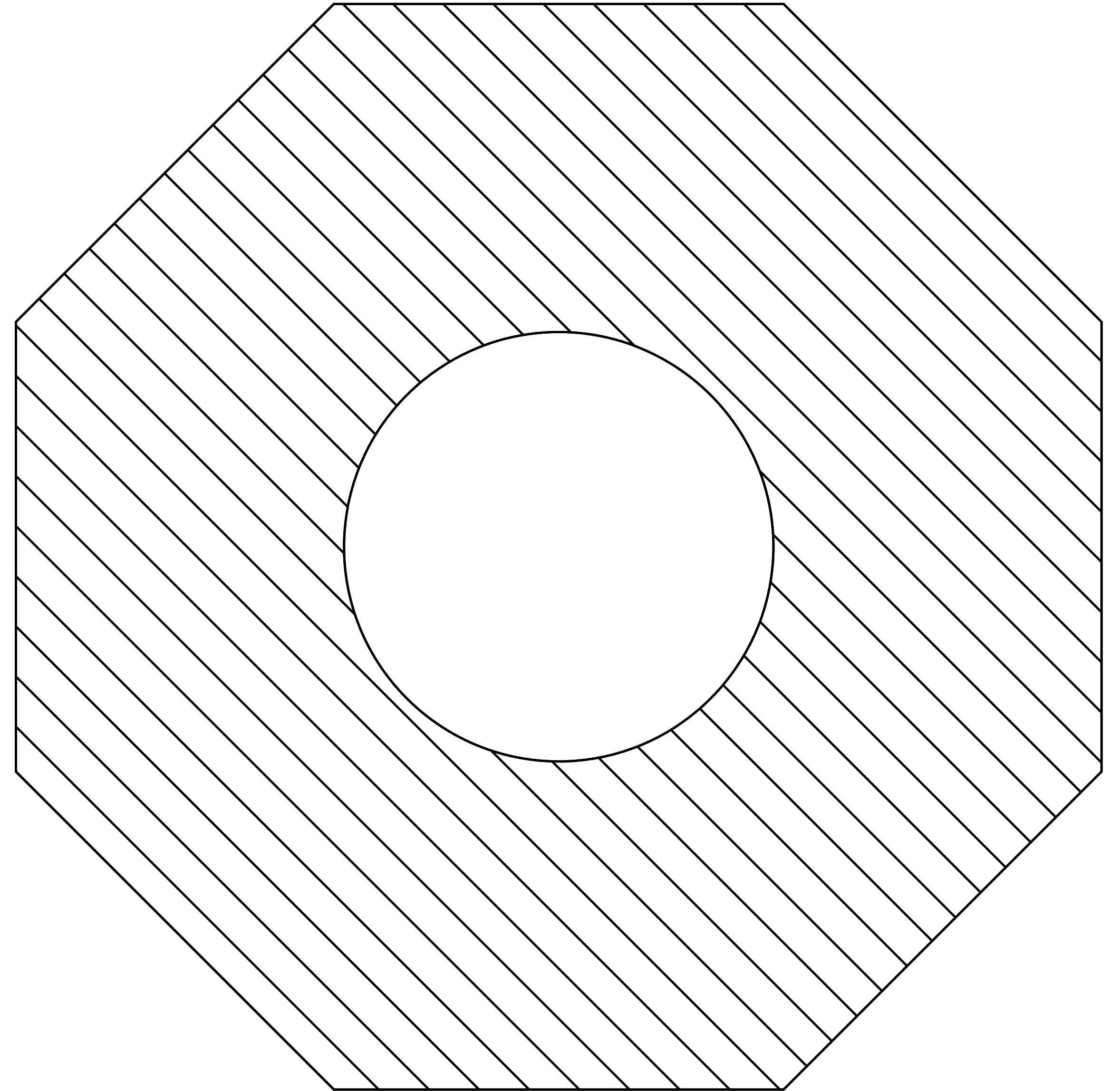


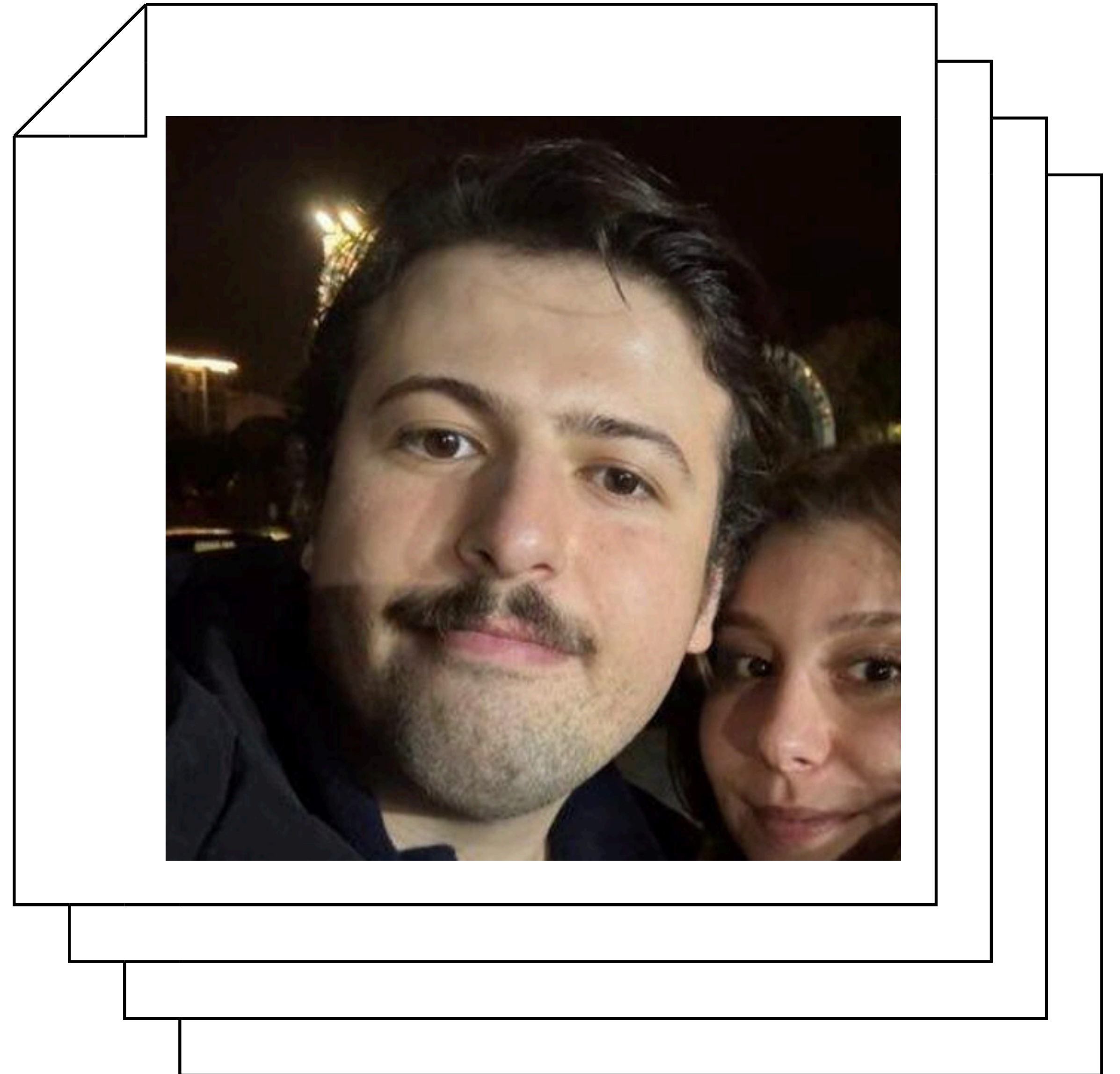
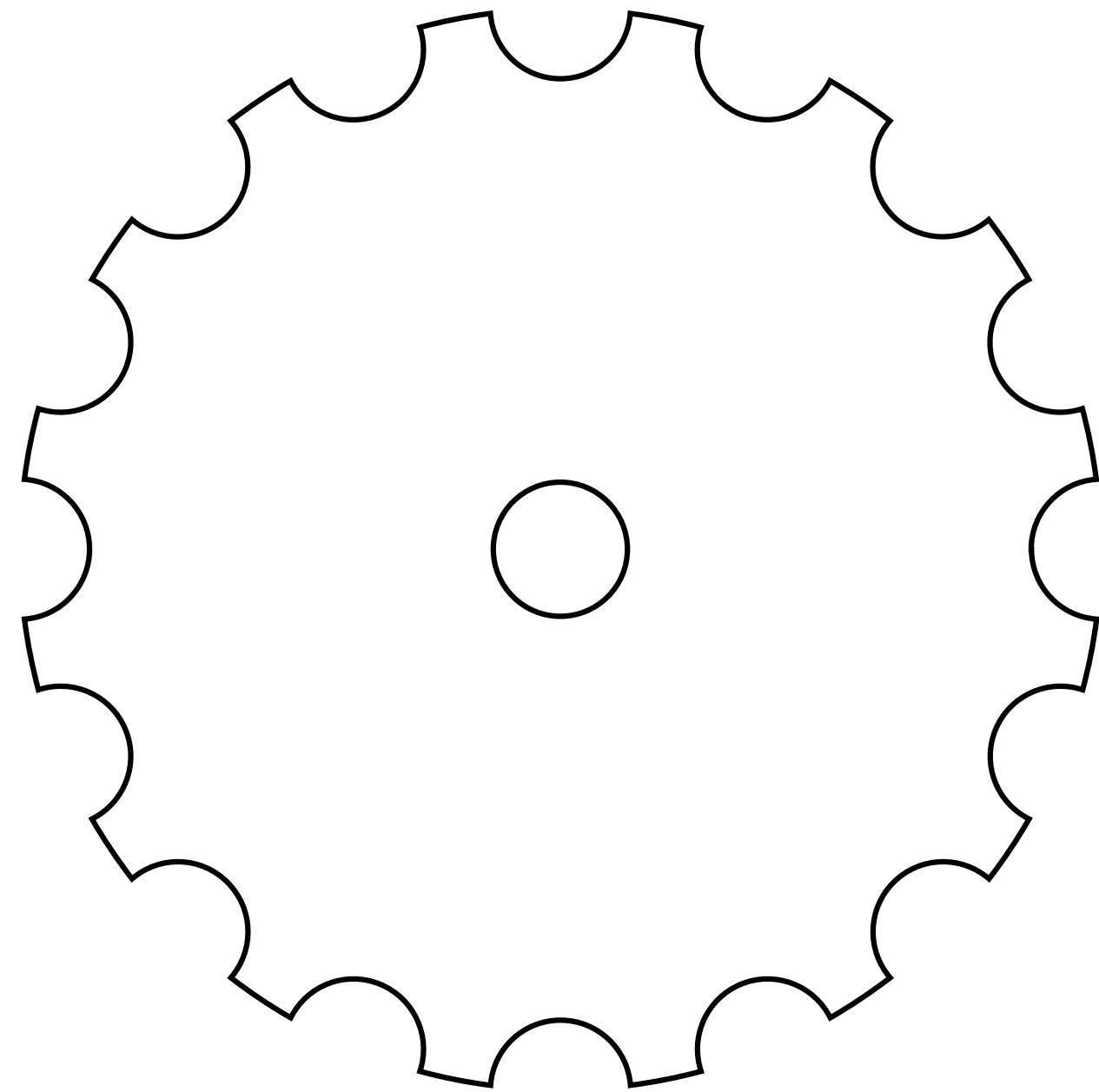
Garbage Collection Mechanisms

Across V8, cppgc
& Native Runtimes



Mert Can Altın

- Node.js Core Contributor
- Trendyol - Software Engineer
- Creator Kiren & Fixify



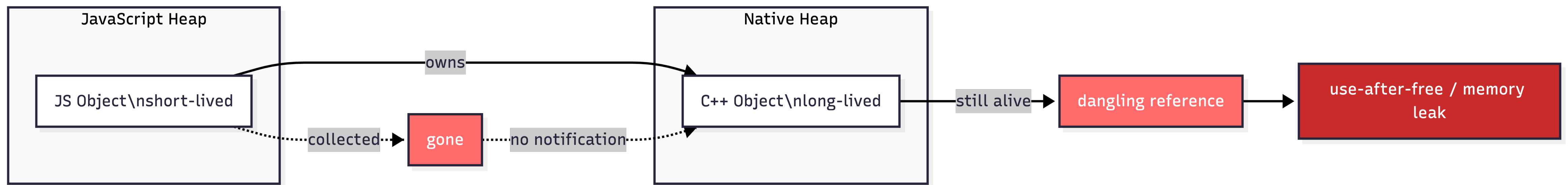
Agenda

- Why is GC Integration Hard?
- V8 & cppgc Fundamentals
- 4 Runtimes, 4 Approaches
- Node.js Evolution
- Future: cppgc Young Gen & Minor GC

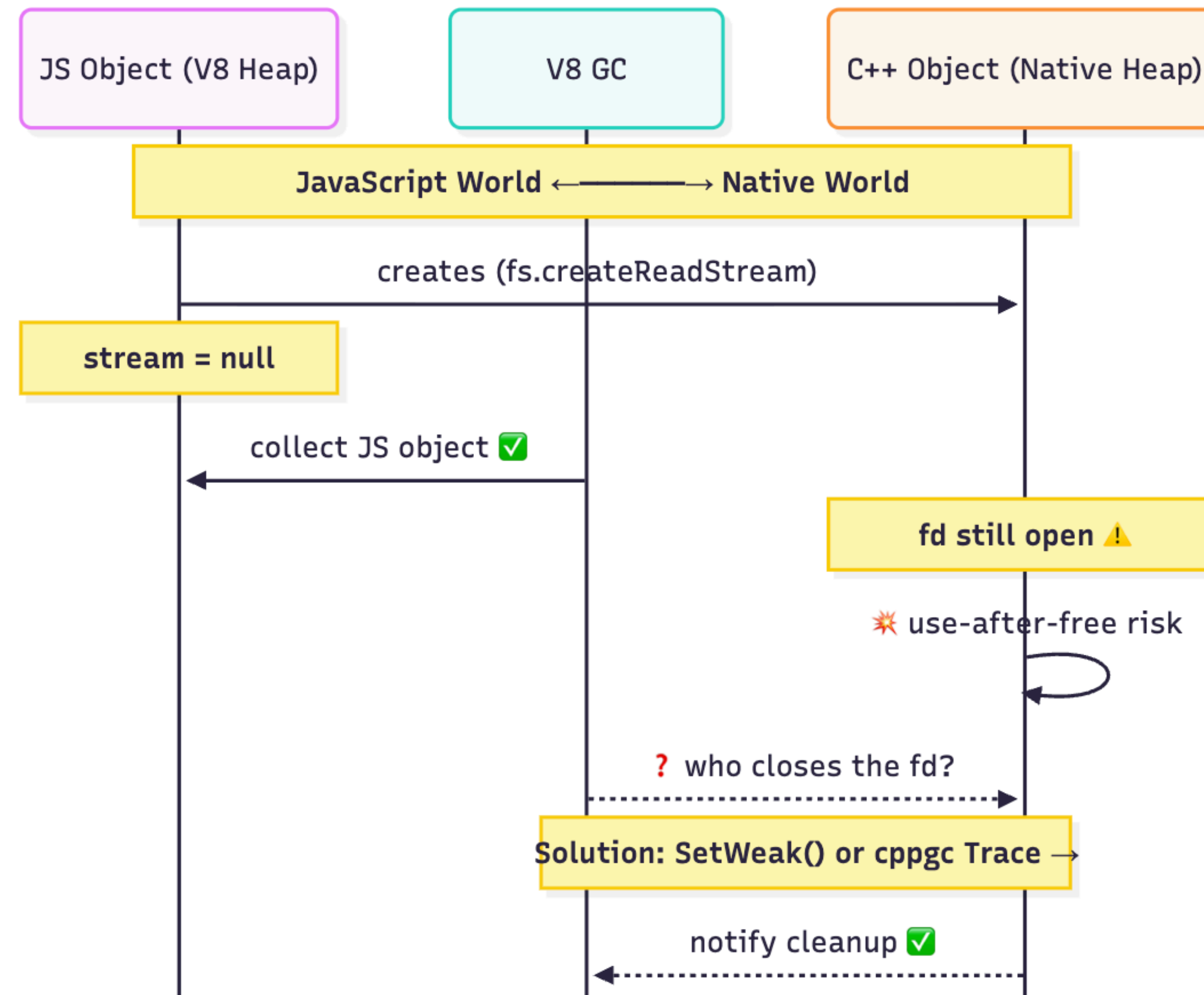
Why is GC Integration Hard?

Two heaps, two lifecycles, zero coordination JS GC doesn't know about native allocations Native objects can outlive their JS wrappers No unified collection trigger across runtimes

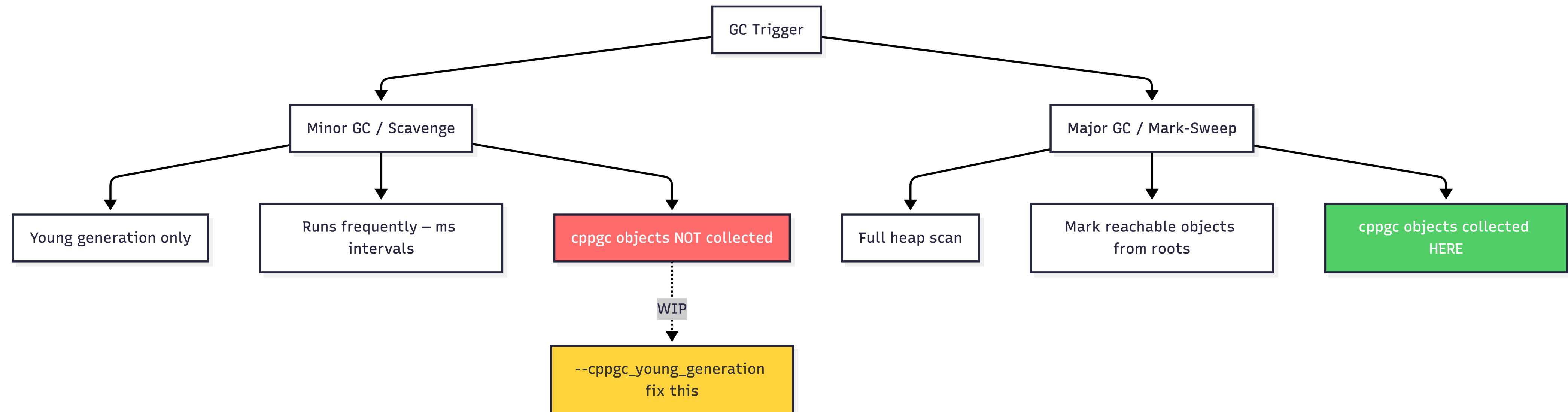
Result: use-after-free, memory leaks, complexity



The Problem: When Two Worlds Collide



How V8 & cppgc actually work together



4 Runtimes, 4 Approaches

Node.js



Deno.js



Cloudflare Workers

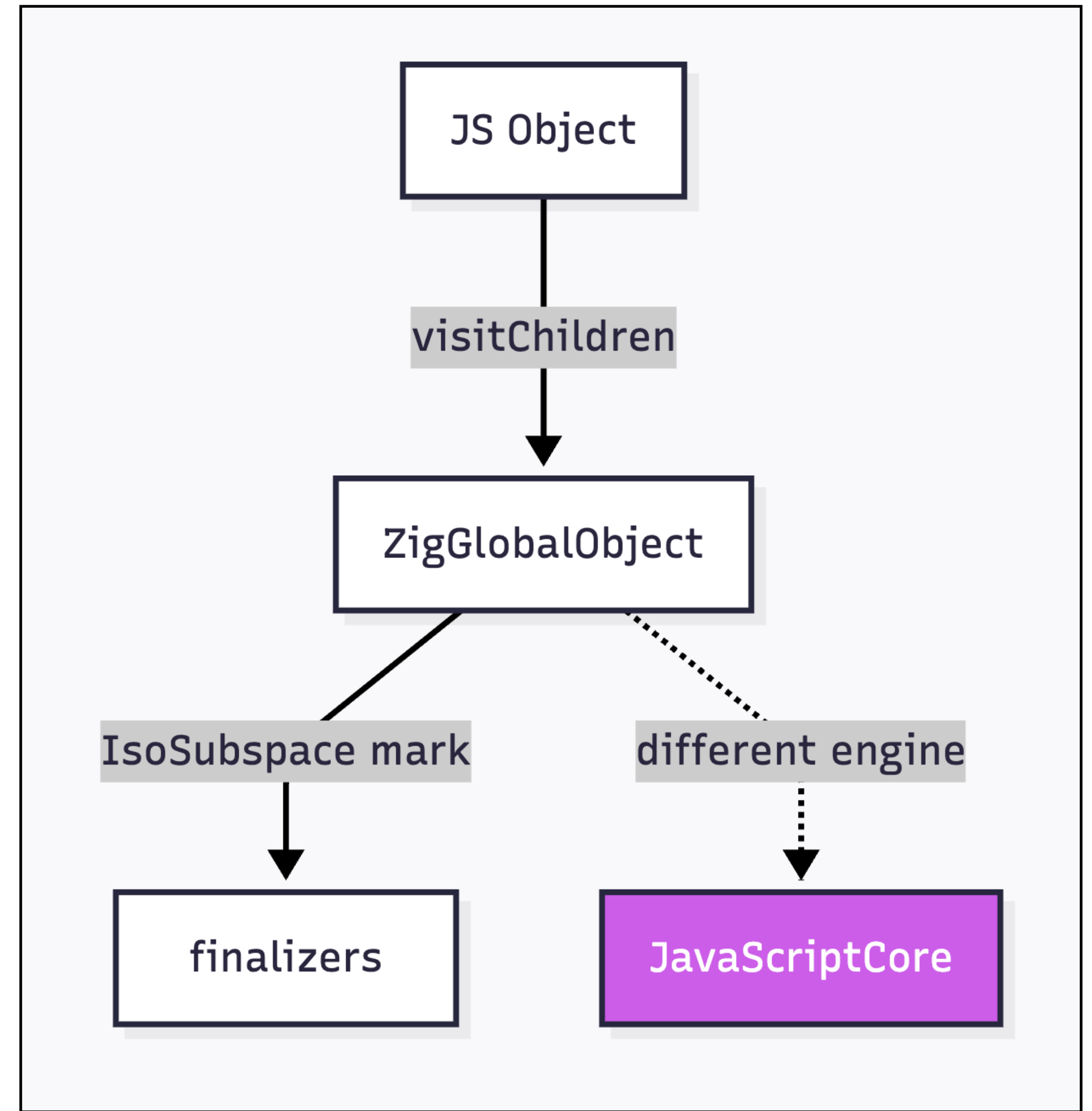


Bun.js



Bun

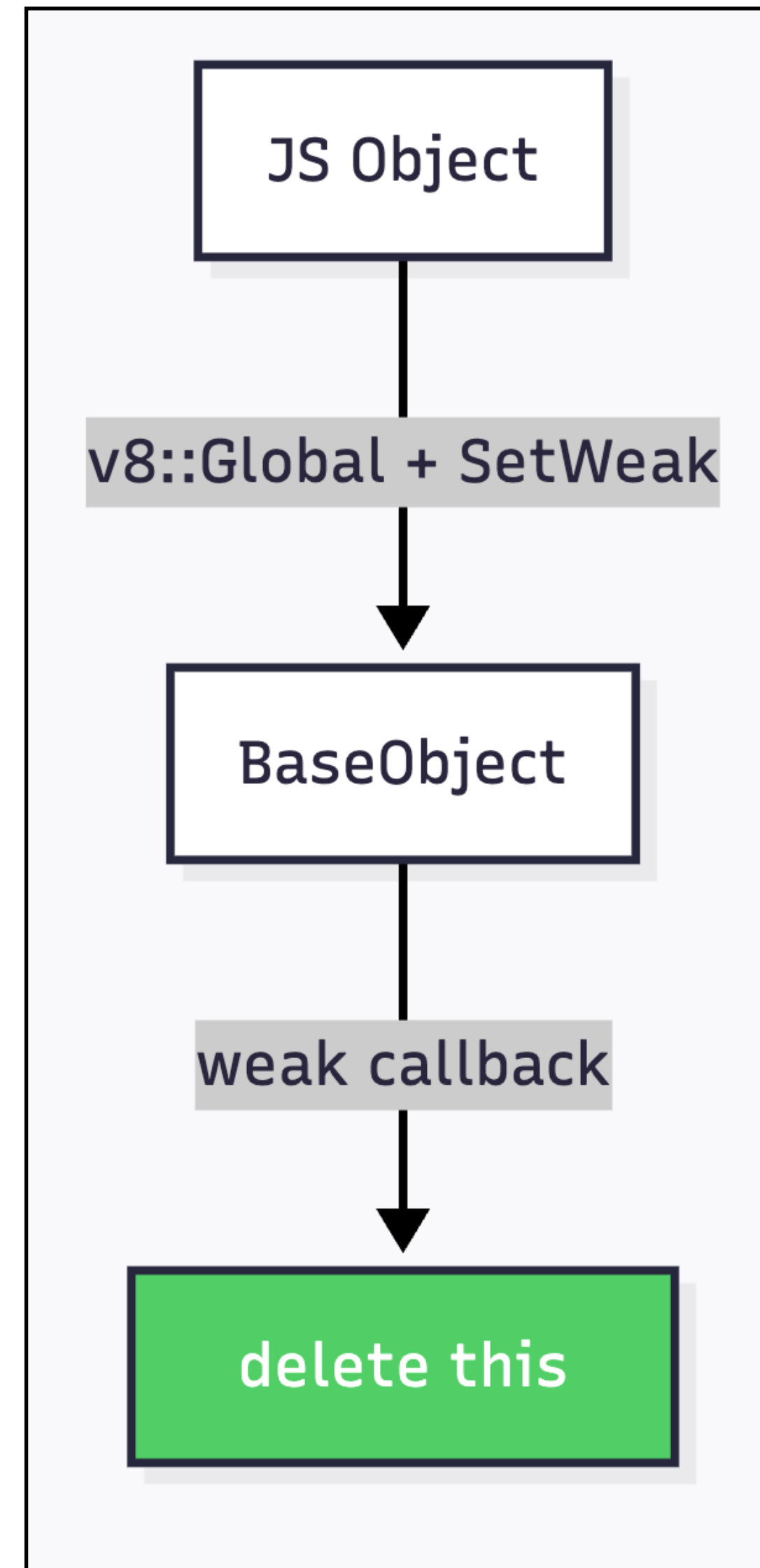
Uses JavaScriptCore instead of V8 fundamentally
different GC visitChildren() method for object
graph tracing IsoSubspace isolates object types
for efficient marking ZigGlobalObject manages
finalizers Generational GC support out of the box



Node.js BaseObject

`persistent_handle_.SetWeak()` gives GC permission to collect V8 GC triggers weak callback when collecting Inside

callback: `persistent_handle_.Reset()` and `OnGCCollect()`
Usually deletes C++ object via `delete this` Manual ref counting, error prone



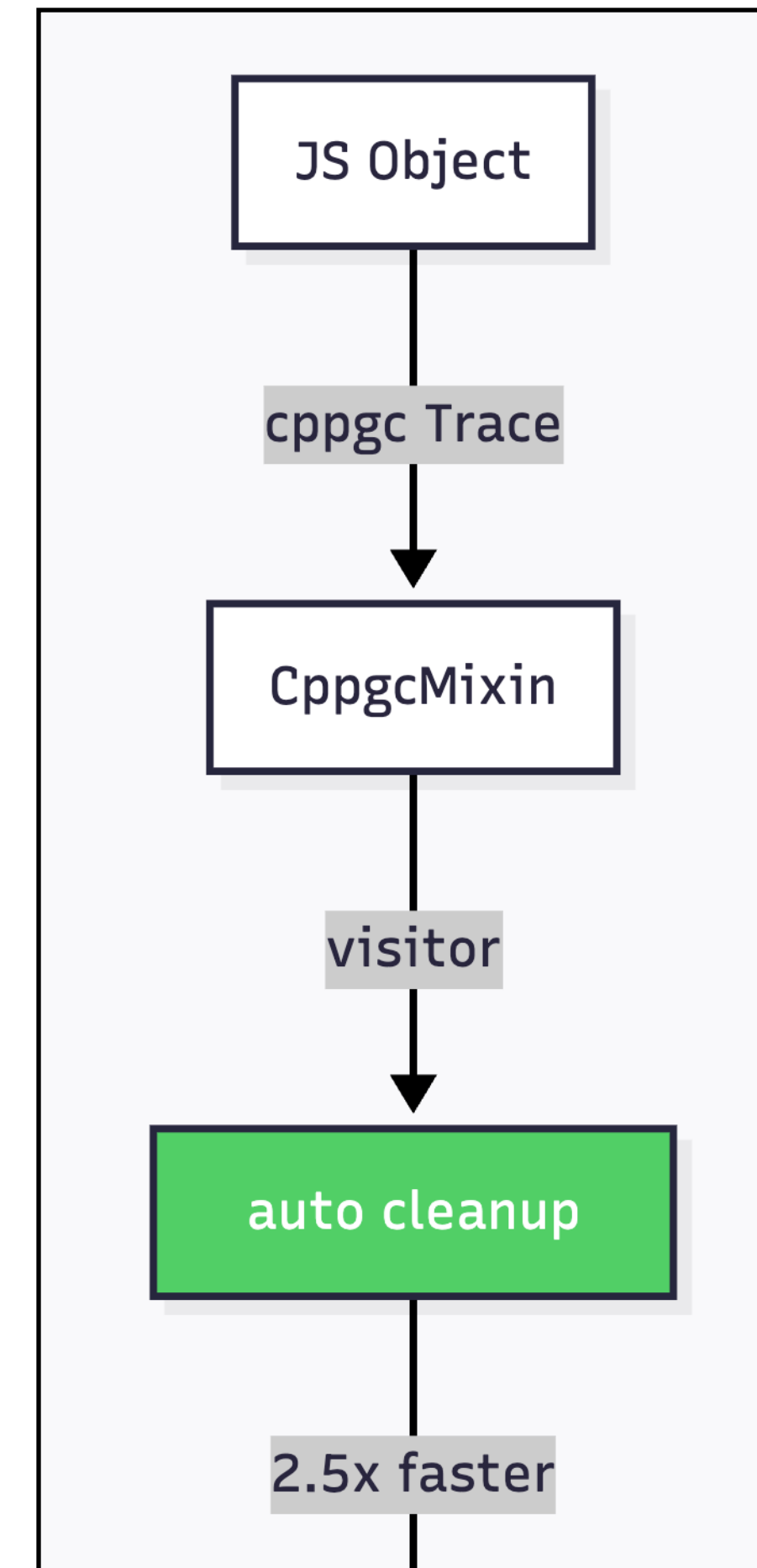
Node.js CppgcMixin

Participates directly in cppgc object graph
Trace() method replaces manual SetWeak
callbacks Visitor pattern, automatic member

tracing Result: 2.5x faster object creation Source:

Joyee Cheung, nodejs/node#51017

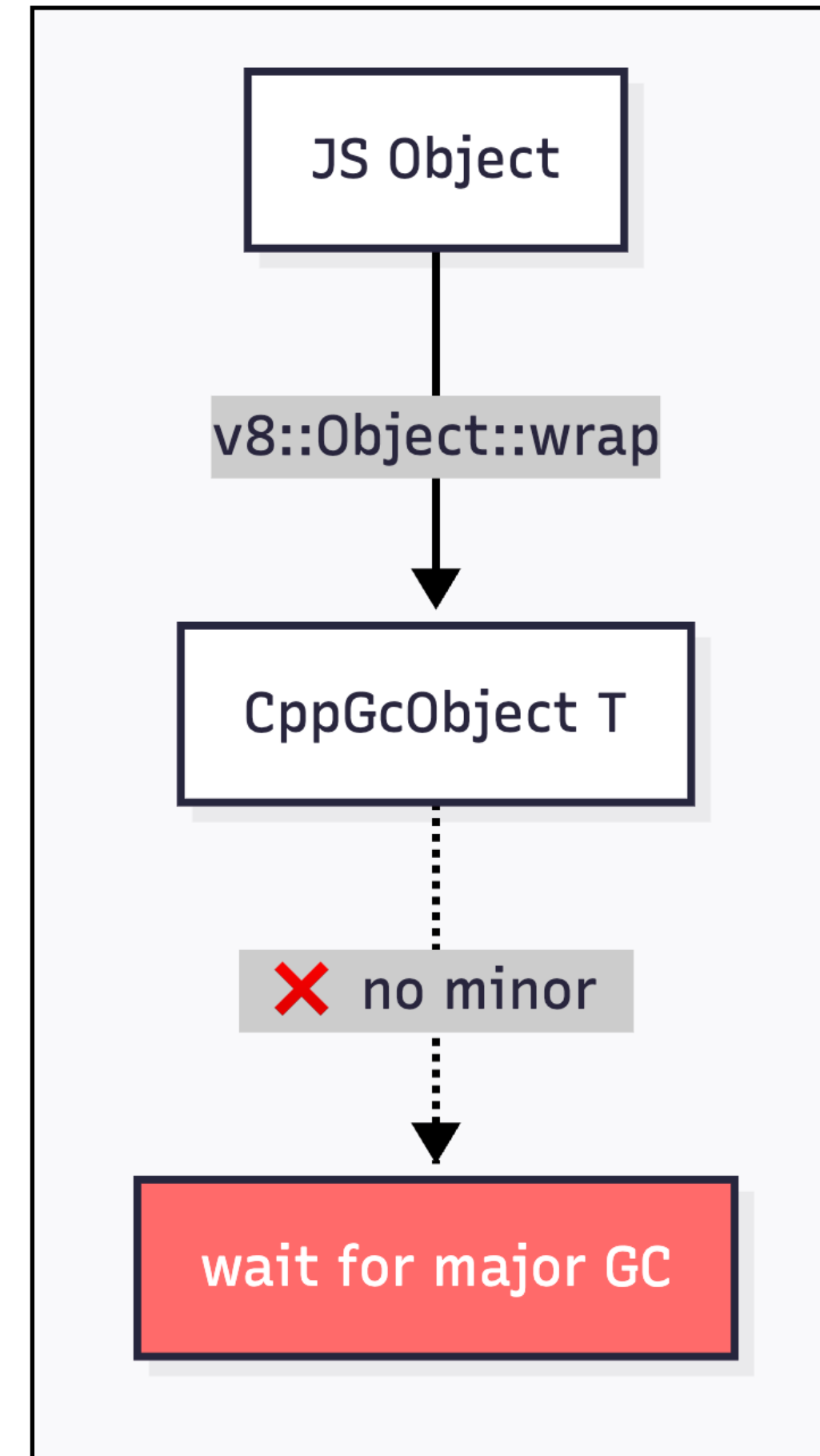
Actively in migration, PR #51017 (draft, blocked on V8 upstream)



Deno

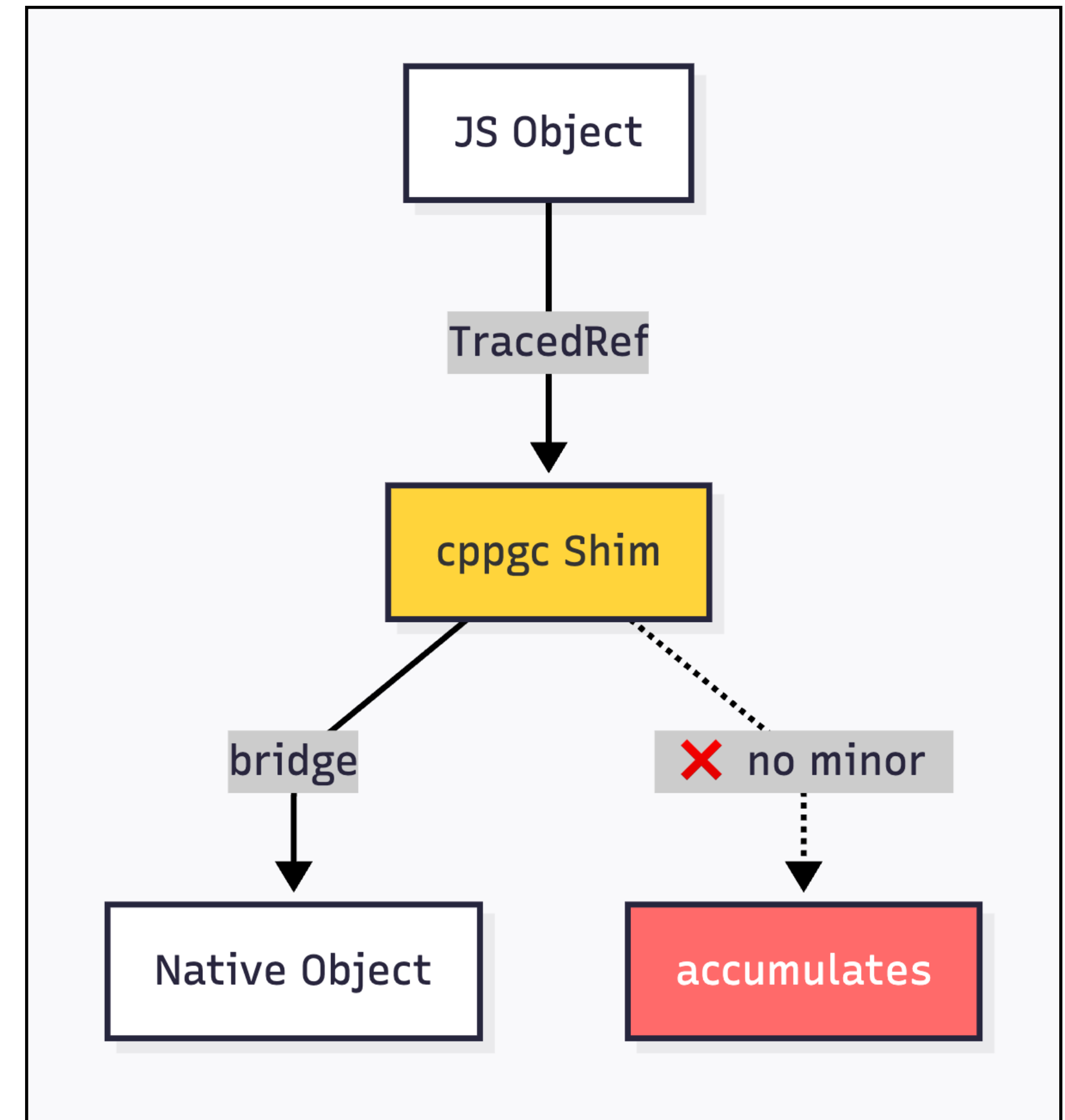
No shim layer, allocates directly on cppgc heap
Cleanest architecture with minimal complexity
Natural fit with Rust ownership model

Limitation: cppgc objects cannot be collected in
minor GC

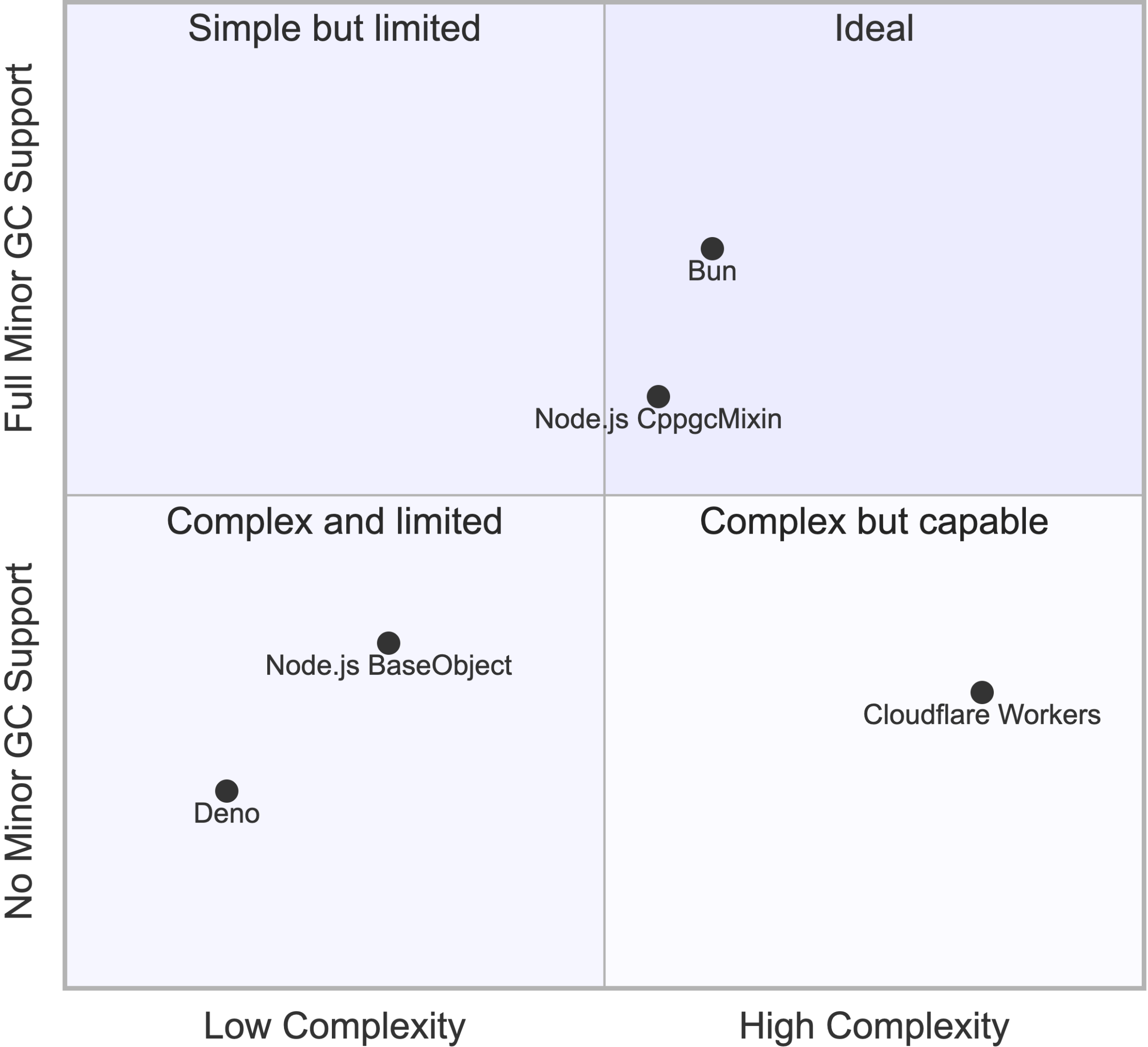


Cloudflare Workers

Written with KJ idioms before cppgc existed, full migration impractical
Uses a shim layer on cppgc heap as bridge to existing C++ objects
Droppable TracedReference lets V8 drop wrappers when unreachable from JS
Shims cannot be collected in minor GC, accumulation until major GC



Runtime Complexity vs Minor GC Support



GC Trace Flow: How Each Runtime Traces Native Objects

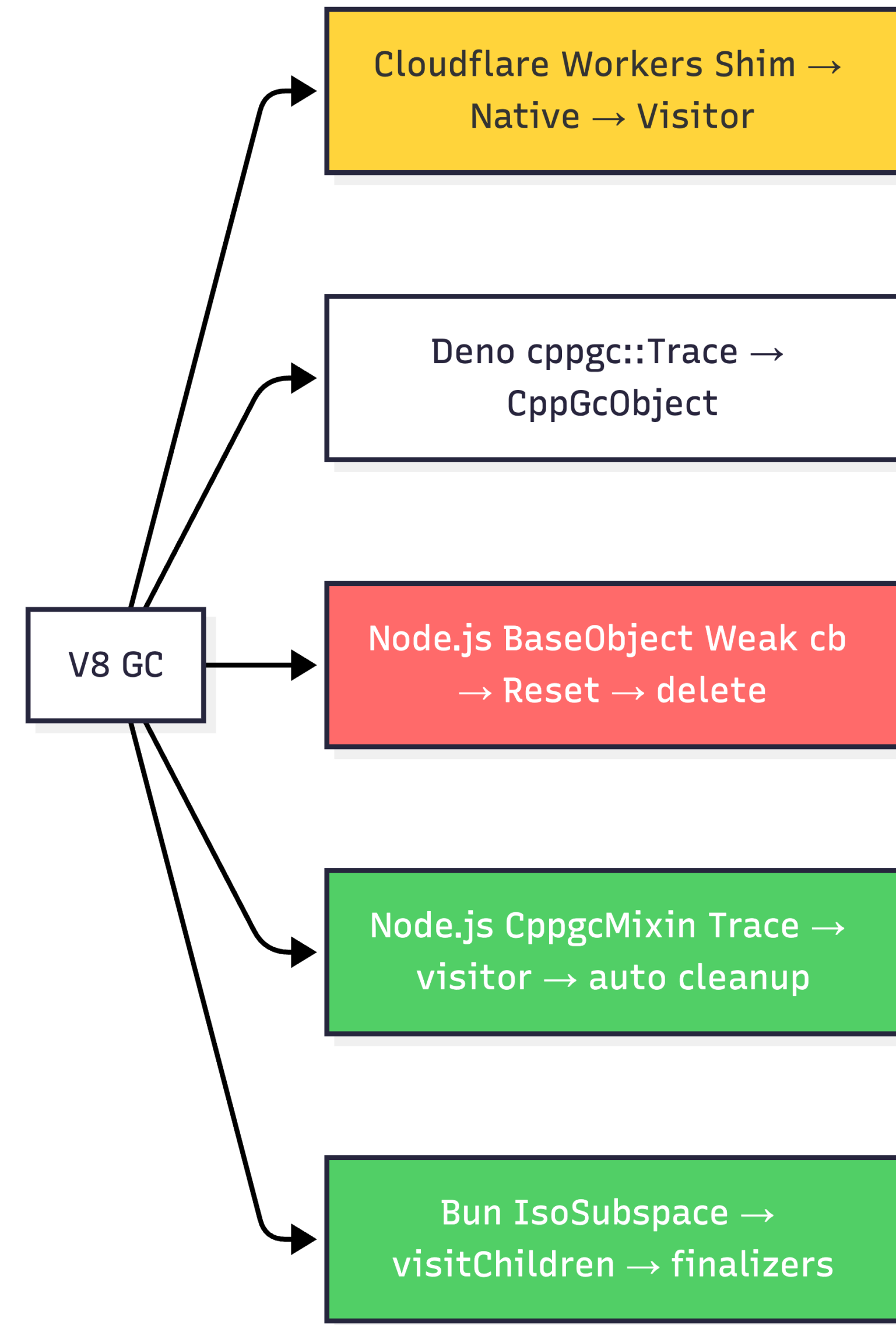
Cloudflare Workers: extra shim layer adds complexity

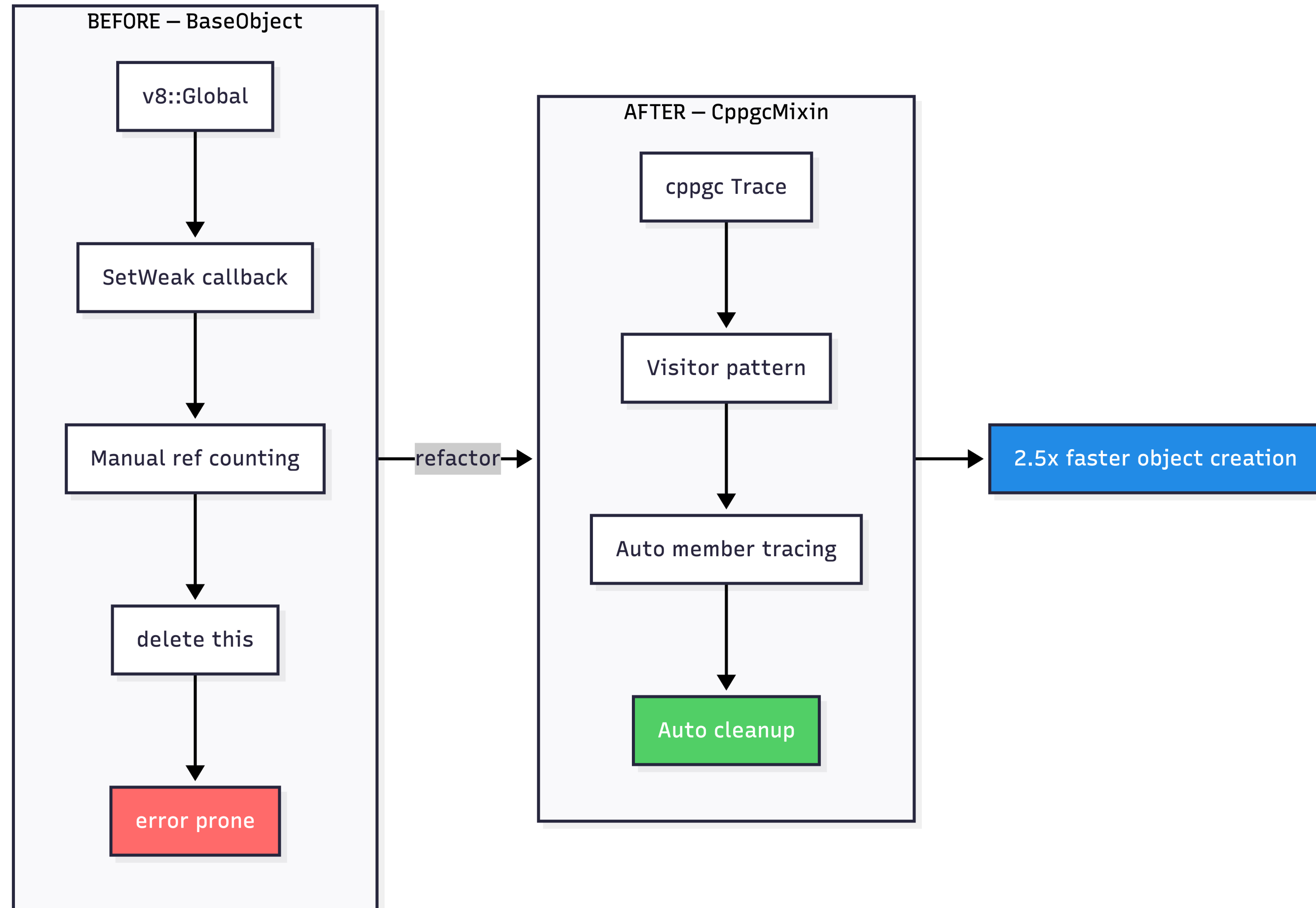
Node.js BaseObject: manual weak callback, error prone

Node.js CppgcMixin: automatic via visitor pattern

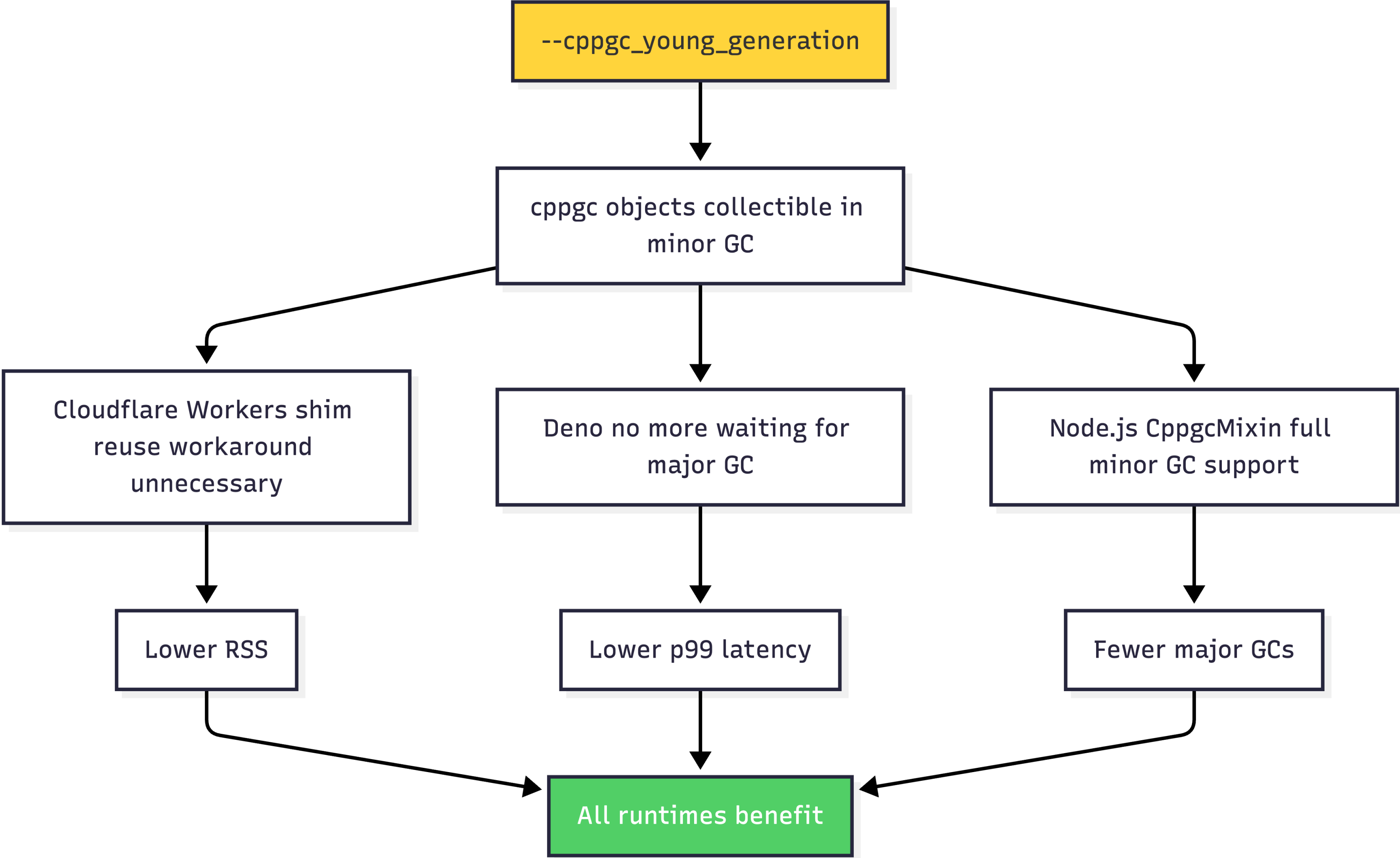
Deno: cleanest path, direct cppgc access

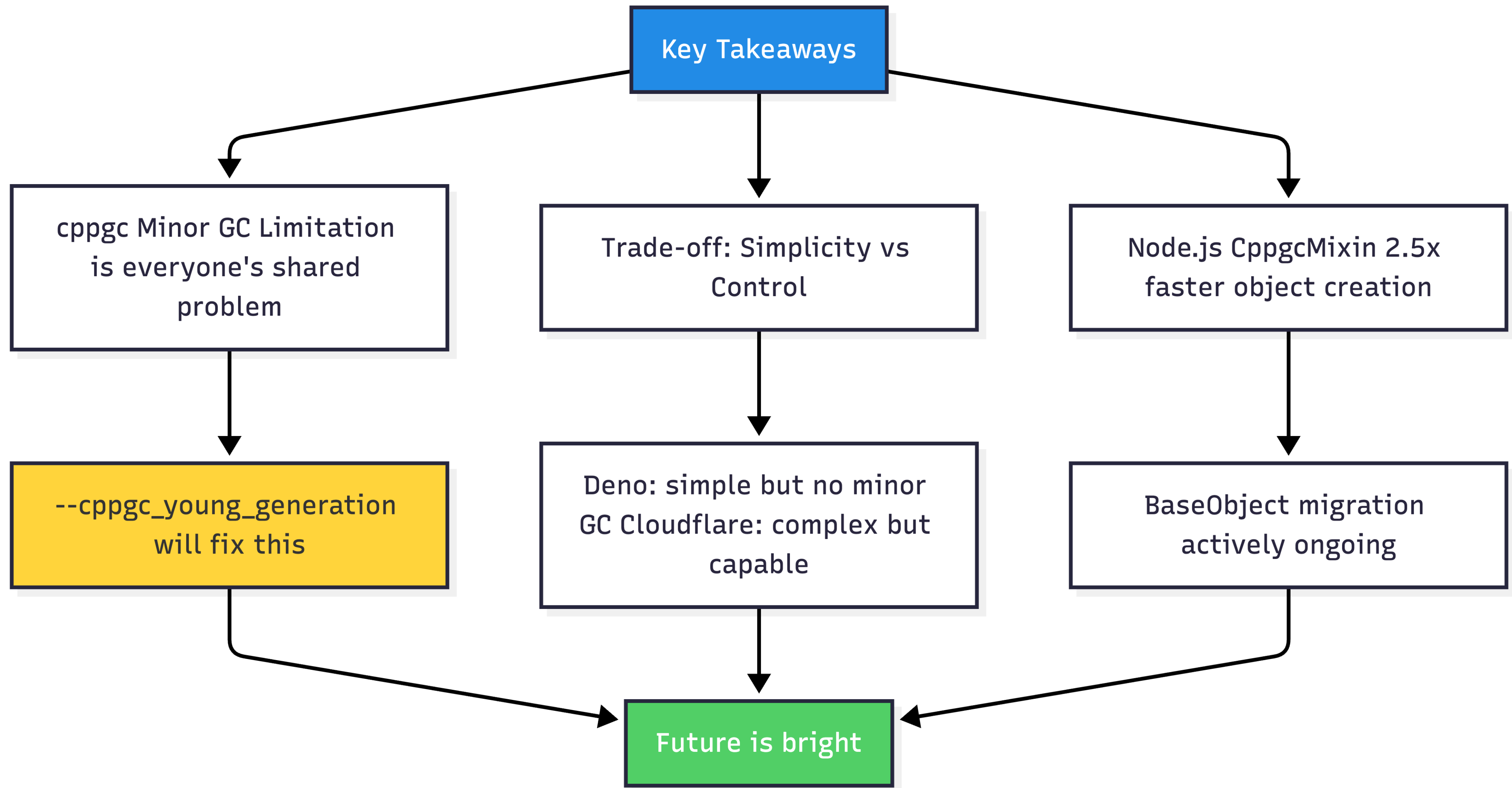
Bun: different engine, same end result





Future: cppgc_young_generation





Thanks for listening

Questions?
github.com/mertcanaltin
Node Congress 2026